

Next Word Prediction Using LSTM

Autocompleting poetry lines using LSTM

This project autocompletes poetry lines related to flowers, using an LSTM-based next-word prediction model.

In []:

In [1]:

```
poems_text = """Some men never think of it.  
You did. You'd come along  
And say you'd nearly brought me flowers  
But something had gone wrong.
```

```
The shop was closed. Or you had doubts —  
The sort that minds like ours  
Dream up incessantly. You thought  
I might not want your flowers.
```

```
It made me smile and hug you then.  
Now I can only smile.  
But, look, the flowers you nearly brought  
Have lasted all this while.
```

```
tell the flowers—they think  
the sun loves them.  
The grass is under the same  
simple-minded impression
```

```
about the rain, the fog, the dew.  
And when the wind blows,  
it feels so good  
they lose control of themselves
```

```
and swobtoggle wildly  
around, bumping accidentally into their  
slender neighbors.  
Forgetful little lotus-eaters,
```

```
solar-powered  
hydroholics, drawing nourishment up  
through stems into their  
thin green skin,
```

```
high on the expensive  
chemistry of mitochondrial explosion,  
believing that the dirt  
loves them, the night, the stars—
```

```
reaching down a little deeper  
with their pale albino roots,  
all Dizzy  
Gillespie with the utter  
sufficiency of everything.
```

```
They don't imagine lawn  
mowers, the four stomachs  
of the cow, or human beings with boots
```

who stop to marvel

at their exquisite
flexibility and color.
They persist in their soft-headed

hallucination of happiness.
But please don't mention it.
Not yet. Tell me
what would you possibly gain

from being right?

Not only the cultivated ones in parks
and gardens, unfolding immaculate petals
on a terrace or trellis, and not just
the wild ones, kissed by elegant birds
in jungle foliage, or brightening roadsides
and meadows, blossoming anyplace that anything
can blossom, but thanks also to flowers
blooming in paintings, on carpets, pottery,
fabrics of dresses and draperies or wherever
the real or invented colors and shapes
of flowers lift the mood of a scene,
as they are snipped from bushes, gathered
in cordless bunches, tied in ribbons
or arranged in rare bouquets for precious vases.
Perfect by nature for gift and centerpiece,
they perfume ballrooms, backyards and prairies,
and, indoors or out the window, they gladden
celebrations and refresh every country
and season, for, even in iciest winter.
The word flower thrives in every language,
adorning what everyone says and imagines
with the beautiful thought of flowers
which teach by timeless example
that life goes by anyway; you might as well
flower.

poem, poem be strong
like a shock wave, Grieg's Concerto in A Minor
put down roots, find the source, bloom, bear fruit
come to life, poem, I need your blood

poem, poem be as perilously lovely
as the drunken woman in the painting by Munch
what counts are only the base colors, yellow, black, red
what counts is fire

there is a time for hope
and a time for despair

what counts is fire
if you have no flesh
you do not know love
nor do you know death

poem, poem be in the sun
in the eye of the world
in the turning of bread into motion
in the constant decay that is the condition of all synthesis
in the blood

fire, be

there is a time for hope
and a time for despair
what counts is fire and ice

poem, poem be like the dark night of the soul

A yellow flower
(Light and spirit)
Sings by itself
For nobody.

A golden spirit
(Light and emptiness)
Sings without a word
By itself.

Let no one touch this gentle sun
In whose dark eye
Someone is awake.

(No light, no gold, no name, no colour
And no thought:
O, wide awake!)

A golden heaven
Sings by itself
A song to nobody.

The bud
stands for all things,
even for those things that don't flower,
for everything flowers, from within, of self-blessing;
though sometimes it is necessary
to reteach a thing its loveliness,
to put a hand on its brow
of the flower
and retell it in words and in touch
it is lovely
until it flowers again from within, of self-blessing;
as Saint Francis
put his hand on the creased forehead
of the sow, and told her in words and in touch
blessings of earth on the sow, and the sow
began remembering all down her thick length,
from the earthen snout all the way
through the fodder and slops to the spiritual curl of the tail,
from the hard spininess spiked out from the spine
down through the great broken heart
to the sheer blue milken dreaminess spurting and shuddering
from the fourteen teats into the fourteen mouths sucking and blowing beneath them:
the long, perfect loveliness of sow.

It doesn't have to be
the blue iris, it could be
weeds in a vacant lot, or a few
small stones; just
pay attention, then patch

a few words together and don't try
to make them elaborate, this isn't
a contest but the doorway

into thanks, and a silence in which
another voice may speak.

Just off the highway to Rochester, Minnesota,
Twilight bounds softly forth on the grass.
And the eyes of those two Indian ponies
Darken with kindness.
They have come gladly out of the willows
To welcome my friend and me.
We step over the barbed wire into the pasture
Where they have been grazing all day, alone.
They ripple tensely, they can hardly contain their happiness
That we have come.
They bow shyly as wet swans. They love each other.
There is no loneliness like theirs.
At home once more,
They begin munching the young tufts of spring in the darkness.
I would like to hold the slenderer one in my arms,
For she has walked over to me
And nuzzled my left hand.
She is black and white,
Her mane falls wild on her forehead,
And the light breeze moves me to caress her long ear
That is delicate as the skin over a girl's wrist.
Suddenly I realize
That if I stepped out of my body I would break
Into blossom.

They are fleeting.
They are fragile.
They require

little water.
They'll surprise you.
They'll remind you

that they aren't
and they are you.

This morning I was walking upstairs

from the kitchen, carrying your
beautiful flowers, the flowers you

brought me last night, calla lilies
and something else, I am not
sure what to call them, white flowers,

of course you had no way of knowing
it has been years since I bought
white flowers—but now you have

and here they are again. I was carrying
your flowers and a coffee cup
and a soft yellow handbag and a book

of poems by a Chinese poet, in
which I had just read the words “come
or go but don’t just stand there

in the doorway,” as usual I was
carrying too many things, you
would have laughed if you saw me.

It seemed especially important
not to spill the coffee as I usually
do, as I turned up the stairs,

inside the whorl of the house as if
I were walking up inside the lilies.
I do not know how to hold all

the beauty and sorrow of my life.

Pluck this little flower and take it, delay not! I fear lest it
droop and drop into the dust.

I may not find a place in thy garland, but honour it with a touch of
pain from thy hand and pluck it. I fear lest the day end before I am
aware, and the time of offering go by.

Though its colour be not deep and its smell be faint, use this flower
in thy service and pluck it while there is time.

Blossom all alone in the chilly snowfall;

You can’t mistake them for snow mass from afar,

For a subtle fragrance keeps coming forth.

Bloom ahead of hundreds of flowers,

```

Appear in the cold of snow and ice.

The frost and the moon are sure to know you,
Being the first to see the face of the spring breeze.


I have no intention of holding onto spring,
I'll leave jealousy for various flowers to endure.
As blossoms fall to the ground and turn to dust,
They leave only the fragrances that never change.


O! my heart now feels so cheerful as I go with footsteps light
    In the daily toil of my dear home;
And I'll tell to you the secret that now makes my life so bright-
    There's a flower at my window in full bloom.
It is radiant in the sunshine, and so cheerful after rain;
    And it wafts upon the air its sweet perfume.
It is very, very lovely! May its beauties never wane-
    This dear flower at my window in full bloom.
Nature has so clothed it in such glorious array,
    And it does so cheer our home, and hearts illumine;
Its dear mem'ry I will cherish though the flower fade away-
    This dear flower at my window in full bloom.
Oft I gaze upon this flower with its blossoms pure and white.
    And I think as I behold its gay costume,
While through life we all are passing may our lives be always bright
    Like this flower at my window in full bloom.
"""

```

In [2]:

```

import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer

tokenizer = Tokenizer()
tokenizer.fit_on_texts([poems_text])

```

In [3]:

```
len(tokenizer.word_index)
```

Out[3]:

648

In [4]:

```
input_sequences = []
for sentence in poems_text.split('\n'):
    tokenized_sentence = tokenizer.texts_to_sequences([sentence])[0]

    for i in range(1, len(tokenized_sentence)):
        input_sequences.append(tokenized_sentence[:i+1])
```

In [5]:

```
#Get the maximum-length:
max_len = max([len(x) for x in input_sequences])
```

In [6]:

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
padded_input_sequences = pad_sequences(input_sequences, maxlen = max_len, padding='pre')
```

In [7]:

```
padded_input_sequences
```

Out[7]:

```
array([[ 0,  0,  0, ...,  0, 185, 186],
       [ 0,  0,  0, ..., 185, 186,  74],
       [ 0,  0,  0, ..., 186,  74,  75],
       ...,
       [ 0,  0,  0, ..., 17,  44,  4],
       [ 0,  0,  0, ..., 44,  4,  73],
       [ 0,  0,  0, ...,  4,  73,  41]], shape=(1282, 16), dtype=int32)
```

In [8]:

```
#Set the input and output variables and apply padding to them:
X = padded_input_sequences[:, :-1]
y = padded_input_sequences[:, -1]

print(X.shape)
print(y.shape)
```

```
(1282, 15)
(1282,)
```

In [9]:

```
#Set the selected word to 1 and all other words as 0 instead of relying on confidence score:
from tensorflow.keras.utils import to_categorical
y = to_categorical(y, num_classes=649)
```

In [10]:

```
y.shape
```

Out[10]:

```
(1282, 649)
```

In []:

In []:

In [11]:

```
#Build the Neural Network:
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

model = Sequential()
model.add(Embedding(649, 100, input_length=12))
model.add(LSTM(150, return_sequences=True))
model.add(LSTM(150))
model.add(Dense(649, activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

C:\Users\Lenovo\anaconda3\Lib\site-packages\keras\src\layers\core\embedding.py:100: UserWarning: Argument `input\_length` is deprecated. Just remove it.
 warnings.warn(

In [12]:

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
lstm_1 (LSTM)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

In [13]:

```
model.fit(X,y,epochs=150)
```

Epoch 1/150
41/41  6s 30ms/step - accuracy: 0.0624 - loss: 6.2421

Epoch 2/150
41/41  1s 30ms/step - accuracy: 0.0655 - loss: 5.8751

Epoch 3/150
41/41  1s 31ms/step - accuracy: 0.0655 - loss: 5.8114

Epoch 4/150
41/41  1s 29ms/step - accuracy: 0.0655 - loss: 5.7941

Epoch 5/150
41/41  1s 29ms/step - accuracy: 0.0655 - loss: 5.7736

Epoch 6/150
41/41  1s 31ms/step - accuracy: 0.0655 - loss: 5.7439

Epoch 7/150
41/41  1s 31ms/step - accuracy: 0.0655 - loss: 5.6266

Epoch 8/150
41/41  2s 38ms/step - accuracy: 0.0671 - loss: 5.5047

Epoch 9/150
41/41  3s 77ms/step - accuracy: 0.0702 - loss: 5.4056

Epoch 10/150
41/41  2s 46ms/step - accuracy: 0.0827 - loss: 5.2827

Epoch 11/150
41/41  2s 39ms/step - accuracy: 0.0788 - loss: 5.1582

Epoch 12/150
41/41  2s 40ms/step - accuracy: 0.0889 - loss: 5.0059

Epoch 13/150
41/41  2s 39ms/step - accuracy: 0.0889 - loss: 4.8577

Epoch 14/150
41/41  1s 34ms/step - accuracy: 0.0967 - loss: 4.7092

Epoch 15/150			
41/41	2s	37ms/step	- accuracy: 0.1100 - loss: 4.5576
Epoch 16/150			
41/41	2s	31ms/step	- accuracy: 0.1154 - loss: 4.4187
Epoch 17/150			
41/41	1s	31ms/step	- accuracy: 0.1318 - loss: 4.2967
Epoch 18/150			
41/41	2s	37ms/step	- accuracy: 0.1373 - loss: 4.1628
Epoch 19/150			
41/41	2s	43ms/step	- accuracy: 0.1529 - loss: 4.0499
Epoch 20/150			
41/41	2s	38ms/step	- accuracy: 0.1661 - loss: 3.9180
Epoch 21/150			
41/41	2s	42ms/step	- accuracy: 0.1747 - loss: 3.8015
Epoch 22/150			
41/41	2s	39ms/step	- accuracy: 0.1911 - loss: 3.6951
Epoch 23/150			
41/41	2s	36ms/step	- accuracy: 0.2067 - loss: 3.5965
Epoch 24/150			
41/41	1s	35ms/step	- accuracy: 0.2324 - loss: 3.4958
Epoch 25/150			
41/41	2s	37ms/step	- accuracy: 0.2473 - loss: 3.3894
Epoch 26/150			
41/41	2s	40ms/step	- accuracy: 0.2769 - loss: 3.2946
Epoch 27/150			
41/41	1s	34ms/step	- accuracy: 0.3034 - loss: 3.1856
Epoch 28/150			
41/41	3s	35ms/step	- accuracy: 0.3307 - loss: 3.0866
Epoch 29/150			
41/41	1s	32ms/step	- accuracy: 0.3619 - loss: 2.9905
Epoch 30/150			
41/41	3s	30ms/step	- accuracy: 0.3752 - loss: 2.9062
Epoch 31/150			
41/41	1s	31ms/step	- accuracy: 0.4111 - loss: 2.8243
Epoch 32/150			
41/41	1s	35ms/step	- accuracy: 0.4321 - loss: 2.7458
Epoch 33/150			
41/41	2s	30ms/step	- accuracy: 0.4555 - loss: 2.6617
Epoch 34/150			
41/41	1s	34ms/step	- accuracy: 0.4735 - loss: 2.5869
Epoch 35/150			
41/41	2s	38ms/step	- accuracy: 0.4922 - loss: 2.5083
Epoch 36/150			
41/41	2s	36ms/step	- accuracy: 0.5109 - loss: 2.4414
Epoch 37/150			
41/41	2s	38ms/step	- accuracy: 0.5273 - loss: 2.3627
Epoch 38/150			
41/41	2s	40ms/step	- accuracy: 0.5390 - loss: 2.2954
Epoch 39/150			
41/41	1s	34ms/step	- accuracy: 0.5546 - loss: 2.2139
Epoch 40/150			
41/41	2s	36ms/step	- accuracy: 0.5757 - loss: 2.1632
Epoch 41/150			
41/41	2s	39ms/step	- accuracy: 0.5874 - loss: 2.1050
Epoch 42/150			
41/41	2s	46ms/step	- accuracy: 0.5944 - loss: 2.0299
Epoch 43/150			
41/41	2s	30ms/step	- accuracy: 0.6092 - loss: 1.9756
Epoch 44/150			
41/41	2s	43ms/step	- accuracy: 0.6248 - loss: 1.9204
Epoch 45/150			
41/41	2s	33ms/step	- accuracy: 0.6396 - loss: 1.8576
Epoch 46/150			
41/41	3s	33ms/step	- accuracy: 0.6560 - loss: 1.8036
Epoch 47/150			
41/41	1s	35ms/step	- accuracy: 0.6599 - loss: 1.7557
Epoch 48/150			
41/41	2s	37ms/step	- accuracy: 0.6677 - loss: 1.7015
Epoch 49/150			
41/41	2s	38ms/step	- accuracy: 0.6763 - loss: 1.6592
Epoch 50/150			
41/41	2s	40ms/step	- accuracy: 0.6895 - loss: 1.6142

Epoch 51/150			
41/41	2s	37ms/step	- accuracy: 0.7044 - loss: 1.5723
Epoch 52/150			
41/41	2s	43ms/step	- accuracy: 0.6997 - loss: 1.5244
Epoch 53/150			
41/41	2s	50ms/step	- accuracy: 0.7215 - loss: 1.4867
Epoch 54/150			
41/41	3s	60ms/step	- accuracy: 0.7301 - loss: 1.4411
Epoch 55/150			
41/41	2s	44ms/step	- accuracy: 0.7348 - loss: 1.4065
Epoch 56/150			
41/41	2s	39ms/step	- accuracy: 0.7434 - loss: 1.3694
Epoch 57/150			
41/41	2s	36ms/step	- accuracy: 0.7488 - loss: 1.3277
Epoch 58/150			
41/41	2s	31ms/step	- accuracy: 0.7605 - loss: 1.2934
Epoch 59/150			
41/41	2s	37ms/step	- accuracy: 0.7598 - loss: 1.2641
Epoch 60/150			
41/41	2s	37ms/step	- accuracy: 0.7676 - loss: 1.2339
Epoch 61/150			
41/41	2s	43ms/step	- accuracy: 0.7746 - loss: 1.1984
Epoch 62/150			
41/41	2s	40ms/step	- accuracy: 0.7769 - loss: 1.1828
Epoch 63/150			
41/41	2s	29ms/step	- accuracy: 0.7761 - loss: 1.1451
Epoch 64/150			
41/41	2s	38ms/step	- accuracy: 0.7824 - loss: 1.1247
Epoch 65/150			
41/41	2s	47ms/step	- accuracy: 0.7917 - loss: 1.0924
Epoch 66/150			
41/41	2s	40ms/step	- accuracy: 0.7988 - loss: 1.0660
Epoch 67/150			
41/41	1s	35ms/step	- accuracy: 0.8034 - loss: 1.0256
Epoch 68/150			
41/41	2s	37ms/step	- accuracy: 0.7964 - loss: 1.0249
Epoch 69/150			
41/41	2s	38ms/step	- accuracy: 0.8003 - loss: 0.9955
Epoch 70/150			
41/41	2s	41ms/step	- accuracy: 0.8112 - loss: 0.9654
Epoch 71/150			
41/41	2s	31ms/step	- accuracy: 0.8050 - loss: 0.9526
Epoch 72/150			
41/41	2s	36ms/step	- accuracy: 0.8112 - loss: 0.9264
Epoch 73/150			
41/41	2s	36ms/step	- accuracy: 0.8183 - loss: 0.8970
Epoch 74/150			
41/41	2s	31ms/step	- accuracy: 0.8214 - loss: 0.8731
Epoch 75/150			
41/41	1s	34ms/step	- accuracy: 0.8307 - loss: 0.8571
Epoch 76/150			
41/41	1s	35ms/step	- accuracy: 0.8261 - loss: 0.8493
Epoch 77/150			
41/41	2s	41ms/step	- accuracy: 0.8292 - loss: 0.8253
Epoch 78/150			
41/41	2s	56ms/step	- accuracy: 0.8354 - loss: 0.8029
Epoch 79/150			
41/41	2s	37ms/step	- accuracy: 0.8245 - loss: 0.8066
Epoch 80/150			
41/41	2s	44ms/step	- accuracy: 0.8432 - loss: 0.7813
Epoch 81/150			
41/41	2s	44ms/step	- accuracy: 0.8409 - loss: 0.7614
Epoch 82/150			
41/41	2s	30ms/step	- accuracy: 0.8479 - loss: 0.7469
Epoch 83/150			
41/41	2s	38ms/step	- accuracy: 0.8495 - loss: 0.7313
Epoch 84/150			
41/41	2s	40ms/step	- accuracy: 0.8502 - loss: 0.7167
Epoch 85/150			
41/41	2s	42ms/step	- accuracy: 0.8588 - loss: 0.7057
Epoch 86/150			
41/41	2s	41ms/step	- accuracy: 0.8557 - loss: 0.6936

Epoch 87/150				
41/41		2s	39ms/step	- accuracy: 0.8549 - loss: 0.6768
Epoch 88/150				
41/41		2s	37ms/step	- accuracy: 0.8565 - loss: 0.6741
Epoch 89/150				
41/41		2s	39ms/step	- accuracy: 0.8526 - loss: 0.6573
Epoch 90/150				
41/41		2s	46ms/step	- accuracy: 0.8643 - loss: 0.6373
Epoch 91/150				
41/41		2s	37ms/step	- accuracy: 0.8580 - loss: 0.6260
Epoch 92/150				
41/41		2s	39ms/step	- accuracy: 0.8596 - loss: 0.6332
Epoch 93/150				
41/41		2s	41ms/step	- accuracy: 0.8612 - loss: 0.6120
Epoch 94/150				
41/41		2s	44ms/step	- accuracy: 0.8674 - loss: 0.5914
Epoch 95/150				
41/41		2s	41ms/step	- accuracy: 0.8682 - loss: 0.5844
Epoch 96/150				
41/41		2s	32ms/step	- accuracy: 0.8697 - loss: 0.5793
Epoch 97/150				
41/41		3s	34ms/step	- accuracy: 0.8752 - loss: 0.5583
Epoch 98/150				
41/41		1s	34ms/step	- accuracy: 0.8651 - loss: 0.5652
Epoch 99/150				
41/41		2s	37ms/step	- accuracy: 0.8721 - loss: 0.5443
Epoch 100/150				
41/41		2s	38ms/step	- accuracy: 0.8775 - loss: 0.5449
Epoch 101/150				
41/41		2s	40ms/step	- accuracy: 0.8768 - loss: 0.5263
Epoch 102/150				
41/41		2s	45ms/step	- accuracy: 0.8799 - loss: 0.5199
Epoch 103/150				
41/41		2s	38ms/step	- accuracy: 0.8736 - loss: 0.5093
Epoch 104/150				
41/41		2s	41ms/step	- accuracy: 0.8736 - loss: 0.4994
Epoch 105/150				
41/41		2s	41ms/step	- accuracy: 0.8752 - loss: 0.4989
Epoch 106/150				
41/41		2s	46ms/step	- accuracy: 0.8822 - loss: 0.4838
Epoch 107/150				
41/41		2s	43ms/step	- accuracy: 0.8814 - loss: 0.4822
Epoch 108/150				
41/41		2s	39ms/step	- accuracy: 0.8846 - loss: 0.4735
Epoch 109/150				
41/41		2s	38ms/step	- accuracy: 0.8814 - loss: 0.4672
Epoch 110/150				
41/41		2s	42ms/step	- accuracy: 0.8861 - loss: 0.4627
Epoch 111/150				
41/41		2s	42ms/step	- accuracy: 0.8869 - loss: 0.4630
Epoch 112/150				
41/41		2s	42ms/step	- accuracy: 0.8846 - loss: 0.4498
Epoch 113/150				
41/41		2s	45ms/step	- accuracy: 0.8892 - loss: 0.4364
Epoch 114/150				
41/41		2s	41ms/step	- accuracy: 0.8877 - loss: 0.4385
Epoch 115/150				
41/41		2s	39ms/step	- accuracy: 0.8892 - loss: 0.4280
Epoch 116/150				
41/41		2s	34ms/step	- accuracy: 0.8885 - loss: 0.4226
Epoch 117/150				
41/41		2s	38ms/step	- accuracy: 0.8877 - loss: 0.4183
Epoch 118/150				
41/41		2s	39ms/step	- accuracy: 0.8846 - loss: 0.4170
Epoch 119/150				
41/41		2s	37ms/step	- accuracy: 0.8885 - loss: 0.4147
Epoch 120/150				
41/41		2s	50ms/step	- accuracy: 0.8908 - loss: 0.4028
Epoch 121/150				
41/41		2s	54ms/step	- accuracy: 0.8892 - loss: 0.4174
Epoch 122/150				
41/41		3s	61ms/step	- accuracy: 0.8892 - loss: 0.3980

Epoch 123/150
41/41  2s 42ms/step - accuracy: 0.8916 - loss: 0.3880
Epoch 124/150
41/41  2s 37ms/step - accuracy: 0.8931 - loss: 0.3863
Epoch 125/150
41/41  2s 42ms/step - accuracy: 0.8916 - loss: 0.3885
Epoch 126/150
41/41  2s 45ms/step - accuracy: 0.8947 - loss: 0.3793
Epoch 127/150
41/41  2s 37ms/step - accuracy: 0.8908 - loss: 0.3785
Epoch 128/150
41/41  2s 47ms/step - accuracy: 0.8931 - loss: 0.3708
Epoch 129/150
41/41  2s 44ms/step - accuracy: 0.8963 - loss: 0.3629
Epoch 130/150
41/41  2s 38ms/step - accuracy: 0.8947 - loss: 0.3601
Epoch 131/150
41/41  2s 48ms/step - accuracy: 0.9025 - loss: 0.3530
Epoch 132/150
41/41  2s 42ms/step - accuracy: 0.8916 - loss: 0.3562
Epoch 133/150
41/41  2s 38ms/step - accuracy: 0.8947 - loss: 0.3497
Epoch 134/150
41/41  2s 46ms/step - accuracy: 0.8939 - loss: 0.3549
Epoch 135/150
41/41  2s 44ms/step - accuracy: 0.8877 - loss: 0.3584
Epoch 136/150
41/41  2s 49ms/step - accuracy: 0.8947 - loss: 0.3461
Epoch 137/150
41/41  3s 79ms/step - accuracy: 0.8908 - loss: 0.3382
Epoch 138/150
41/41  2s 55ms/step - accuracy: 0.8947 - loss: 0.3348
Epoch 139/150
41/41  2s 45ms/step - accuracy: 0.8955 - loss: 0.3355
Epoch 140/150
41/41  2s 42ms/step - accuracy: 0.8955 - loss: 0.3289
Epoch 141/150
41/41  2s 39ms/step - accuracy: 0.8963 - loss: 0.3243
Epoch 142/150
41/41  2s 43ms/step - accuracy: 0.8978 - loss: 0.3218
Epoch 143/150
41/41  2s 51ms/step - accuracy: 0.8924 - loss: 0.3201
Epoch 144/150
41/41  2s 54ms/step - accuracy: 0.8892 - loss: 0.3282
Epoch 145/150
41/41  2s 47ms/step - accuracy: 0.8924 - loss: 0.3209
Epoch 146/150
41/41  2s 40ms/step - accuracy: 0.8970 - loss: 0.3141
Epoch 147/150
41/41  2s 40ms/step - accuracy: 0.8970 - loss: 0.3110
Epoch 148/150
41/41  2s 46ms/step - accuracy: 0.8924 - loss: 0.3090
Epoch 149/150
41/41  2s 40ms/step - accuracy: 0.8955 - loss: 0.3133
Epoch 150/150
41/41  2s 38ms/step - accuracy: 0.8986 - loss: 0.3070

Out[13]:

<keras.src.callbacks.history.History at 0x16974977620>

In [14]:

```
#Make predictions:
import time
import numpy as np
text = "The shop was closed.".lower()

for i in range(10):
    # tokenize
    token_text= tokenizer.texts_to_sequences([text])[0]
    # padding
```

```

padded_token_text = pad_sequences([token_text], maxlen=56, padding='pre')
# predict
pos = np.argmax(model.predict(padded_token_text))

for word, index in tokenizer.word_index.items():
    if index == pos:
        text = text + " " + word
        print(text)
        time.sleep(2)

```

```

1/1 _____ 1s 546ms/step
the shop was closed. or
1/1 _____ 0s 69ms/step
the shop was closed. or you
1/1 _____ 0s 67ms/step
the shop was closed. or you brought
1/1 _____ 0s 72ms/step
the shop was closed. or you brought of
1/1 _____ 0s 76ms/step
the shop was closed. or you brought of the
1/1 _____ 0s 66ms/step
the shop was closed. or you brought of the condition
1/1 _____ 0s 62ms/step
the shop was closed. or you brought of the condition of
1/1 _____ 0s 57ms/step
the shop was closed. or you brought of the condition of holding
1/1 _____ 0s 72ms/step
the shop was closed. or you brought of the condition of holding onto
1/1 _____ 0s 59ms/step
the shop was closed. or you brought of the condition of holding onto spring

```

In []:

In [15]:

```

#Export the model and the tokenizer:
import pickle

with open('flower_poetry_lstm.pkl', 'wb') as f:
    pickle.dump(model, f)

with open('tokenizer.pkl', 'wb') as f:
    pickle.dump(tokenizer, f)

```

In []:

PROJECT BY:

- **Name:** Om Satyawan Pathak
- **Email:** omsatyawanpathakgit@gmail.com (or) omsatyawanpathakwebdevelopment@gmail.com
- **Linkedin:** www.linkedin.com/in/om-satyawan-pathak-029b02368

In []: